Project Proposal and Problem Definition

Project: PhishGuard Trainer (working name) · a phishing-awareness *simulator*
Team: Takım 16 (5 students)

- Ali Kaan Demiralp (230205034)
- Murat Yatağan (220205005)
- Furkan Berkay Öztürk (210204021)
- Duhan Kaan Şenol (220205054)
- Erol Yağız Gemalmaz (220204057)

Course: SENG 321 Software Engineering · Sprint 1

Role	Team member
Lead Software Architect and Technical Lead; DevOps and CI/CD Engineer	Ali Kaan Demiralp
Product Owner	Murat Yatağan
Scrum Master and Agile Process Coach	Furkan Berkay Öztürk
Requirements and UX Engineer	Duhan Kaan Şenol
Quality Assurance and Test Automation Engineer	Erol Yağız Gemalmaz

Role details and the RACI matrix are in `04-Team-Roles-and-RACI.md`.

---

1. Problem evidence

Affected people: everyday email users (students, staff, small-office employees) who are not security professionals.

Current situation: Phishing is the most common entry point for account takeover. Awareness is usually taught through a one-off slide deck or a yearly PDF. People read it once, never practice, and never get feedback on *why* a specific message was suspicious.

Consequences: compromised accounts, credential reuse across services, financial loss, and an IT team that cleans up the same mistake repeatedly.

Supporting sources. The following publications report on phishing as an attack vector and are used as problem evidence:

- Verizon, *Data Breach Investigations Report* (annual), https://www.verizon.com/business/resources/reports/dbir/ : identifies phishing and the human element among the leading causes of breaches.
- APWG, *Phishing Activity Trends Report* (quarterly), https://apwg.org/trendsreports/ : tracks the volume of phishing attacks.

Measurable baseline: no practice tool exists for the target audience today, so the baseline is each new user's *first-session score*. Improvement is measured against it.

Limitation: the project does not run field studies or send real phishing email. Evidence of effectiveness is limited to in-application score improvement across sessions; the project does not claim a reduction in real-world phishing incidents.

---|---|---|---|
Learner (end user)	Practise spotting phishing, see mistakes explained, track progress	Primary user	Uses the trainer
Trainer / Admin	Add and edit scenarios, see aggregate results	Operational user	Maintains content
Course instructor	Evaluates engineering process and product	Sponsor / grader	Governance
Team (5 students)	Delivers the system	Builder	Implements
Security community	Source of public phishing indicators (OWASP, CERT guidance)	External	Knowledge source

3. Proposed outcome (solution hypothesis)

If learners practise on realistic *fake* email scenarios and get immediate feedback pointing at the exact red flags, their accuracy on later sessions will rise.

Success measures (all measurable inside the product):

1. A new user registers, completes a 10-scenario session and sees a score in under 5 minutes.
2. Average session score of returning users on their 3rd session is higher than on their 1st (hypothesis, measured from the application's own session data).
3. An admin adds a new scenario in under 2 minutes without touching code.

The hypothesis can change if the data contradicts it.

4. Feasibility

Area	Assessment
Technical	Standard web app (SPA + REST API + relational DB). All team skills are covered by well-known open-source stacks. Low risk.
Data	No personal data beyond username/email and score. Scenario content is written by us (fictional brands and domains).
Schedule	10 weeks, 5 people. MVP at Week 8. Scope below is deliberately small.
Legal / ethical	Simulator only. Scenarios use fictional or reserved domains (`example.com` , `.test`). No real brand logos, no real links, no sending of email. See risks R3.
Operational	One `docker compose up` brings everything up (course requirement).

5. Release scope

Included in release 1 (MVP):

- Registration, login, roles (`learner` , `admin`).
- Library of fake email scenarios (phishing or legitimate, with red-flag explanations).
- Training session: show scenario, learner decides *phishing / legitimate* (optionally marks suspicious elements), gets instant feedback.
- Scoring per session and score history per user.
- Admin dashboard: scenario CRUD, aggregate statistics (sessions, average score, hardest scenarios).
- Security basics: password hashing, role-based access control, input validation, rate limiting.

Explicitly out of scope:

- Sending real or simulated phishing mail to real people.
- Email-provider integration, browser plug-ins, mobile app.
- Gamification beyond a simple score (no leaderboard, badges are a stretch goal).
- Multi-tenant organisations, SSO, payments, localisation.
- Field testing with real users (not required by the course spec).

6. Assumptions

#	Assumption	How we validate
A1	30 scenarios are enough for a convincing demo	Content review by team members; adjust count
A2	Learners understand English scenarios	Keep language plain; i18n is out of scope
A3	Instructor accepts a simulator without field testing	The project specification contains no field-testing requirement
A4	Team can host a demo via Docker on one laptop or cheap VM	Smoke test in Sprint 3

7. Risks

#	Risk	Likelihood	Impact	Response
R1	Uneven team contribution	High	High	Small tasks, every story has an owner, PR reviews visible, escalate to instructor early with Git evidence
R2	Scope creep (gamification, fancy UI)	Medium	Medium	Backlog is prioritised; stretch items only after MVP
R3	Scenario content could be misused as a real phishing kit	Low	Medium	Fictional brands only, no real links, README states the educational purpose, admin-only content editing
R4	Weak security in a *security* product (embarrassing in the defense)	Medium	High	Security stories are in the MVP backlog, not optional

#	Risk	Likelihood	Impact	Response
R5	Late integration between UI and API	Medium	Medium	API-first: OpenAPI contract + mock server from Week 3
R6	Hosting / Docker surprises in demo	Medium	High	CI builds the image on every PR; rehearse demo from clean machine

8. Open-source license decision record

Decision: Apache License 2.0 for all original code, and the same license for the scenario dataset we author.

Rationale

- Permissive: others (including commercial users) may use, modify and redistribute.
- Explicit patent grant, which MIT lacks. This matters a little for a security tool and is the main reason we prefer it over MIT.
- Does not force derivative works to be open (unlike GPL), keeping reuse by other teams or companies easy.
- Widely understood, and compatible with most dependencies we are likely to use (MIT, BSD, ISC, Apache-2.0).

Alternatives considered

- MIT: simpler text but no patent grant. Acceptable fallback.
- GPL-3.0: would force anyone embedding our trainer to open-source theirs. Not our goal; also complicates dependency compatibility.

Follow-up actions

1. Add full text as `LICENSE` and a short `NOTICE` file in the repo root.
2. Run a dependency licence scan in CI (e.g. `license-checker` or `pip-licenses`) and record unknown or copyleft (GPL/AGPL) dependencies as release risks.
3. Do not copy third-party phishing templates or brand assets without checking their licence.
4. Follow the university policy on student intellectual property. This record is not legal advice.

9. Validation actions (before Sprint 2)

- Team walkthrough: each of the 5 members completes a draft session and reports confusion points.
- Role model: the five core roles of the specification are applied; the Product Owner and Requirements Engineer role is divided between a Product Owner and a Requirements and UX Engineer so that each of the four team members besides the Lead Architect holds one main role.
- Collect 10 real-style phishing examples from public sources (rewritten, not copied) to seed the scenario library.